PROJECT REPORT

Pearls AQI Predictor

AI-Powered 3-Day Air Quality Index Forecasting System for Karachi, Pakistan

---

Prepared by

Kashfi Karim Ali

Data Science Project

Repository: github.com/kashfi2312192/karachi-aqi-predictor
Live Dashboard: karachi-aqi-predictor-2026.streamlit.app

July - September 2026

Table of Contents

1. Introduction

Air pollution is one of the most pressing environmental challenges faced by rapidly urbanizing cities, and Karachi — Pakistan's largest metropolitan area — is no exception. Rising vehicular emissions, industrial activity, construction dust, and seasonal weather patterns cause the city's Air Quality Index (AQI) to fluctuate significantly from day to day. These fluctuations directly affect public health, outdoor activity planning, and policy decisions, yet residents rarely have access to reliable, forward-looking air quality information.

The Pearls AQI Predictor project was undertaken to address this gap by building a complete, end-to-end machine learning system capable of forecasting Karachi's AQI for the next three days. The project combines historical air quality observations with meteorological data, applies systematic feature engineering, trains and compares multiple machine learning algorithms, and deploys the final solution as a live, interactive web application. Beyond raw prediction, the project also focuses on model transparency through explainability techniques (SHAP) and on validating model behaviour through diagnostic analysis, so that the forecasts can be trusted and interpreted rather than treated as a black box.

This report documents the complete data science workflow followed in the project — from problem framing and data preparation, through model development and evaluation, to explainability, diagnostics, and deployment.

2. Background

The Air Quality Index (AQI) is a standardized scale used worldwide to communicate how polluted the air currently is, or how polluted it is forecast to become, and what associated health effects might be a concern. It is derived from concentrations of pollutants such as PM2.5, PM10, NO_2, SO_2, O_3, and CO, and mapped onto a scale with descriptive categories (Good, Moderate, Unhealthy for Sensitive Groups, Unhealthy, Very Unhealthy, and Hazardous).

Traditional AQI reporting in many developing cities, including Karachi, is largely descriptive of current conditions rather than predictive. Most publicly available tools report today's AQI but do not forecast how it will evolve over the following days. Since AQI is influenced by time-lagged and weather-dependent patterns — such as wind speed, humidity, temperature, and prior pollution levels — it is well suited to being modeled as a supervised time-series forecasting problem using machine learning.

Machine learning offers a practical way to capture these non-linear relationships between weather, historical pollution levels, and future AQI values. By treating AQI forecasting as a regression problem over multiple prediction horizons (1, 2, and 3 days ahead), it becomes possible to learn horizon-specific patterns and select the modelling approach that performs best for each horizon, rather than forcing a single model to generalize across all of them.

3. Problem Statement

Residents of Karachi currently lack access to a reliable, forward-looking, and locally-relevant AQI forecasting tool. Existing sources mostly report present-day air quality without providing a short-term outlook, which limits the public's ability to plan outdoor activities, take health precautions, or otherwise respond proactively to worsening air quality.

The problem addressed by this project can be stated as follows:

- Given historical AQI observations and weather data for Karachi, design a machine learning system that accurately forecasts AQI values 1, 2, and 3 days into the future.
- Determine which machine learning algorithm performs best for each individual forecast horizon, rather than assuming one model fits all horizons equally well.
- Ensure the resulting predictions are explainable and diagnostically sound, so that both technical and non-technical users can trust the forecast.
- Deploy the solution as an accessible, real-time web application that updates automatically as new data becomes available.

4. Objectives

- Predict Karachi's AQI for the next 3 days (t+1, t+2, t+3).
- Use historical pollution and weather information as predictive features.
- Engineer time-series features (lags, recency, temporal patterns) to improve forecasting performance.
- Train and compare multiple machine learning algorithms across all three horizons.
- Select the best-performing, horizon-specific model for production use.
- Store and manage engineered features using a feature store (Hopsworks).
- Register, version, and track trained models.
- Perform model evaluation and diagnostic analysis to validate model behaviour.
- Explain model predictions using SHAP.
- Automate daily data processing and prediction generation.
- Expose predictions through a REST API and an interactive dashboard.

5. Dataset and Feature Engineering

5.1 Data Sources

The project uses historical AQI and weather data for Karachi sourced from the Open-Meteo API, which provides both air quality and meteorological observations. This combination allows the models to learn how weather conditions (such as temperature, humidity, and wind) relate to changes in pollutant concentration and the resulting AQI.

5.2 Feature Engineering

Raw AQI and weather readings alone are not sufficient for accurate multi-day forecasting, so a dedicated feature engineering stage was built into the pipeline. This stage transforms raw time-series observations into a structured feature set suitable for supervised learning, including:

- Historical AQI values as base predictors.
- Lagged observations capturing recent trends in pollution levels.
- Recency-based features that emphasize the influence of the most recent readings.
- Temporal features capturing daily and seasonal patterns.
- Weather-derived predictors (temperature, humidity, wind, etc.).

These engineered features allow the models to learn the relationship between current environmental conditions and future AQI values, rather than relying on raw, unprocessed readings.

5.3 Feature Storage

Engineered features are stored and version-controlled using Hopsworks, a feature store platform. This ensures consistency between the features used during training and those used during inference, and supports the project's automated daily prediction pipeline.

6. Methodology

The project follows a standard end-to-end machine learning workflow, structured as a repeatable pipeline rather than a one-off notebook exercise. The overall system architecture is summarized below.

6.1 System Architecture

- Data Collection — Historical AQI and weather data retrieved from Open-Meteo.
- Data Processing — Cleaning, structuring, and preparing the AQI and weather data.
- Feature Engineering — Generating lag, recency, temporal, and weather-derived features.
- Feature Storage — Persisting engineered features in the Hopsworks feature store.
- Model Training & Evaluation — Training multiple regression algorithms per forecast horizon and comparing performance.
- Model Selection — Choosing the best model independently for Day 1, Day 2, and Day 3 forecasts.
- Prediction Pipeline — Generating the latest 3-day forecast (latest_predictions.csv).
- Serving Layer — A Flask REST API (port 5000) exposing predictions.
- Presentation Layer — An interactive Streamlit dashboard (port 8501) for end users.

6.2 Forecasting Approach

Rather than building a single model to predict all three days at once, AQI forecasting was framed as three related but independent regression problems — one for each horizon (t+1, t+2, t+3). This horizon-specific approach allows each model to specialize in the error characteristics of its own prediction window, since uncertainty naturally increases the further ahead the forecast extends.

7. Model Development

A wide range of machine learning algorithms were implemented and evaluated for AQI regression, covering linear, tree-based/ensemble, kernel-based, and boosting methods. This comparative approach ensured that model selection was driven by empirical performance rather than a single default choice.

7.1 Algorithms Evaluated

Model	Category	Purpose in this Project
Ridge Regression	Linear (Regularized)	Baseline linear model with L2 regularization to control overfitting.
Lasso / ElasticNet	Linear (Regularized)	Linear baselines with L1/L1+L2 regularization for feature sparsity.
Extra Trees Regressor	Ensemble (Bagging)	Randomized tree ensemble to capture non-linear feature interactions.
Random Forest / Multi-Output RF	Ensemble (Bagging)	Robust tree ensemble baseline for non-linear patterns.
Gradient Boosting	Ensemble (Boosting)	Sequential boosting model to reduce residual error iteratively.
XGBoost	Ensemble (Boosting)	Optimized gradient boosting for high accuracy and efficiency.
CatBoost	Ensemble (Boosting)	Boosting model handling categorical/temporal features robustly.
Support Vector Regression (SVR)	Kernel-based	Captures non-linear relationships via kernel functions.

Table 1: Machine learning algorithms evaluated during model development.

7.2 Training Strategy

Each algorithm was trained separately for each of the three forecast horizons (t+1, t+2, t+3), using the engineered feature set derived from historical AQI and weather data. Preprocessing steps such as StandardScaler and RobustScaler were applied where required by the algorithm (particularly for SVR and Ridge), and models were trained and validated on chronologically consistent train/test splits appropriate for time-series data.

8. Model Confirmation and Selection Process

Confirming which model to use in production was based on a systematic, metric-driven comparison rather than a single model being chosen upfront. The confirmation process followed these steps:

- Train every candidate algorithm (Ridge, Extra Trees, Gradient Boosting, XGBoost, CatBoost, Random Forest, SVR, Lasso, ElasticNet, LSTM, Prophet) independently for each forecast horizon.
- Evaluate each trained model on held-out test data using three standard regression metrics: RMSE (Root Mean Squared Error), MAE (Mean Absolute Error), and R^2 (coefficient of determination).
- Compare models within each horizon separately, since the best-performing algorithm for Day 1 is not necessarily the best for Day 2 or Day 3.
- Select the model with the lowest error (RMSE/MAE) and highest R^2 for each horizon as the final, production-confirmed model for that horizon.
- Register the confirmed models and their metrics in the model registry (Hopsworks Model Registry) for versioning and reproducibility.

This horizon-by-horizon comparison is what ultimately confirmed the final production models — it was not assumed in advance which algorithm would win, and each candidate (including Ridge, Extra Trees, Gradient Boosting, and XGBoost) was given an equal, like-for-like evaluation before selection.

9. Final Production Models and Rationale

Based on the evaluation process described above, different algorithms were confirmed as the best performers for different forecast horizons. The final production configuration is summarized below.

Horizon	Selected Model	RMSE	MAE	R^2
Day 1 (t+1)	SVR + StandardScaler + Recency	13.6674	9.3747	0.7137
Day 2 (t+2)	SVR + RobustScaler	18.3090	13.5716	0.4830
Day 3 (t+3)	CatBoost Regressor	19.7632	15.4517	0.4000

Table 2: Final confirmed production models by forecast horizon.

The rationale for horizon-specific selection is directly tied to the results: the Day-1 model achieves the strongest fit ($R^2 \approx 0.71$) because near-term AQI is more strongly correlated with the most recent observations. As the forecast horizon extends to Day 2 and Day 3, predictive uncertainty naturally increases and error metrics worsen for every algorithm — including Ridge, Extra Trees, Gradient Boosting, and XGBoost — which is why the system uses independently confirmed, horizon-specific models rather than a single generalized model.

While linear models such as Ridge Regression provided fast, interpretable baselines, and ensemble methods such as Extra Trees, Gradient Boosting, and XGBoost captured non-linear feature interactions effectively, the kernel-based SVR and boosting-based CatBoost ultimately produced the lowest error and highest explanatory power on the held-out evaluation data for their respective horizons, which is why they were confirmed as the final production models.

10. Model Explainability — SHAP Analysis

To ensure the forecasting system is not a black box, SHAP analysis was applied to the trained models. SHAP is a game-theoretic technique that attributes each individual prediction to the contribution of each input feature, making it possible to understand which factors are driving the AQI forecast.

10.1 What SHAP Was Used For

- Generating global feature importance rankings to identify which features most influence AQI predictions overall.
- Producing SHAP summary plots to visualize the direction and magnitude of each feature's effect.
- Identifying the Top-20 most influential features across the engineered feature set.
- Performing horizon-specific SHAP analysis, including a dedicated Day-3 CatBoost SHAP study.

10.2 Key Takeaways

The SHAP analysis confirmed that lagged AQI values and recency-based features consistently rank among the most influential predictors, aligning with the intuitive expectation that recent air quality strongly conditions near-future air quality. Weather-derived features contributed additional explanatory power, especially for the longer 2- and 3-day horizons where recent AQI alone becomes a weaker signal. These findings not only validate the feature engineering choices made earlier in the pipeline but also make the model's behaviour interpretable to non-technical stakeholders.

11. Model Diagnostics

Beyond aggregate accuracy metrics, model diagnostics were carried out to check that each confirmed model behaves reliably and consistently, rather than simply reporting a good average score. The diagnostic checks performed include:

- Residual analysis — examining prediction errors (actual − predicted) to check for systematic bias or heteroscedasticity.
- Actual vs. predicted comparison — visually and numerically comparing forecasted AQI against true observed values across the test set.
- Cross-horizon consistency checks — confirming that error grows in a reasonable, expected pattern from Day 1 to Day 3 rather than erratically.
- Metric triangulation — cross-checking RMSE, MAE, and R^2 together, rather than relying on a single metric, to avoid misleading conclusions from any one measure.
- Scaler sensitivity checks — validating that the chosen preprocessing (StandardScaler vs. RobustScaler) genuinely improved, rather than masked, model performance for SVR.

These diagnostics gave confidence that the selected models generalize reasonably to unseen data and that their errors behave in an expected, explainable way rather than indicating a flawed pipeline or leakage.

12. System Deployment

12.1 Feature and Model Management

Hopsworks is used as the central feature store and model registry for the project. It manages engineered features for training and inference and stores registered, versioned models for each forecast horizon, enabling reproducible retraining and rollback if needed.

12.2 Automated Pipelines (GitHub Actions)

The project's data and modelling pipelines are fully automated using scheduled GitHub Actions workflows (cron jobs), removing the need for any manual intervention to keep the system up to date:

- Hourly Feature Pipeline — A GitHub Actions cron job runs every hour to fetch the latest weather and AQI data, regenerate engineered features, and push updates to the Hopsworks feature store.
- Daily Training Pipeline — A separate GitHub Actions cron job runs once a day to retrain candidate models on the latest feature data, re-evaluate performance per horizon, and update the registered production models when appropriate.

This automation ensures that the 3-day AQI forecast is always generated from fresh data, and that the underlying models stay current without requiring manual retraining.

12.3 Prediction Pipeline and API

Following each pipeline run, the system produces fresh 3-day predictions (latest_predictions.csv). These predictions are served through a Flask REST API, deployed on Render, which exposes the current forecast programmatically for consumption by the dashboard or other clients.

12.4 Interactive Dashboard

The final predictions and health categories are presented to end users through an interactive Streamlit dashboard, deployed and publicly accessible online. The dashboard provides an accessible, non-technical view of the 3-day AQI forecast for Karachi.

13. Results Summary

The table below summarizes the current forecast output generated by the deployed system at the time of reporting.

Forecast Horizon	Predicted AQI	Category
Day 1	67.41	Moderate
Day 2	69.81	Moderate
Day 3	79.78	Moderate

Table 3: Example live forecast generated from the deployed pipeline.

Overall, the system demonstrates that near-term (Day 1) AQI forecasting can be performed with strong reliability ($R^2 \approx 0.71$), while medium-term forecasts (Day 2 and Day 3) remain informative but carry increasing uncertainty — a realistic and expected outcome for multi-day environmental forecasting.

14. Conclusion and Future Work

The Pearls AQI Predictor project successfully delivers a complete, end-to-end machine learning system for forecasting Karachi's Air Quality Index up to three days in advance. By evaluating a broad range of algorithms — including Ridge Regression, Extra Trees, Gradient Boosting, XGBoost, CatBoost, Random Forest, and SVR — across multiple forecast horizons, the project confirmed horizon-specific models based on rigorous, metric-driven comparison rather than a single default choice. The addition of SHAP-based explainability and structured model diagnostics ensures the system's predictions are not just accurate, but also interpretable and trustworthy.

Potential directions for future improvement include:

- Extending the forecast horizon beyond 3 days as more historical data accumulates.
- Exploring ensemble/stacking approaches that combine multiple algorithms per horizon.

15. Project Links

- GitHub Repository: github.com/kashfi2312192/karachi-aqi-predictor
- Live Dashboard: karachi-aqi-predictor-2026.streamlit.app